

Negotiated exit from a deployed eUTxO escrow

Reachability, fee incidence, and the bargaining set of the Masumi `vested_pay` validator

Brume

TOKEN2049 Origins, Cardano track • 6–7 October 2026

Abstract

A deployed Plutus V3 validator holds payments between autonomous agents on Cardano. We ask a question its interfaces do not answer: from a given escrow, what can each party do, and which outcomes are reachable without a third party.

We derive the validator’s guard table from its source, then check it against the deployed bytecode by submitting transactions. The main structural result is that, once the seller’s deadline for delivering has passed, a seller’s concession is irreversible and removes arbitration for good. It leaves the buyer as the only party able to move the escrow at all, and the only branch that takes value out of the script constrains no output.

Two consequences follow. Almost no split is self-enforcing, so almost none is reachable by incentives alone. And every split becomes reachable by construction, through a transaction pair whose second leg is signed before the first is submitted. We then price the bargaining set from measured arbitration history rather than from assumption.

We also state what the construction does not do. It removes a refusal. It does not remove a race.

All measurements are on Cardano mainnet and preprod, 6 October 2026. The two networks carry the identical script hash, so every preprod execution interrogates the same bytes that hold the mainnet value.

1 Setting and notation

An escrow is a UTxO $u = (V, d)$ at the validator’s script address. V is a value: a family $(q_j)_{j \in J}$ of quantities indexed by asset class J , lovelace included, which we write $j = L$. d is an inline datum of sixteen fields. We need nine of them.

symbol	field	meaning
b, s	buyer, seller	payment credentials
σ	state	$\sigma \in \Sigma = \{\text{FL}, \text{RS}, \text{RR}, \text{D}\}$
h	result_hash	a byte string. $h = \varepsilon$ denotes empty
c	collateral_return_lovelace	a lovelace floor returned to the buyer on one branch
t_0	unlock_time	the seller’s lower gate on withdrawal
t_1	submit_result_time	the seller’s delivery deadline
t_2	external_dispute_unlock_time	the moment arbitration opens
θ_s, θ_b	seller_cooldown_time, buyer_cooldown_time	per-party lower gates
A, k	admin_vks, required_admins_multi_sig	the arbitration quorum, k of $ A $

A spend presents a redeemer $a \in \mathcal{A}$, where

$$\mathcal{A} = \{a_0 \text{ Withdraw}, a_1 \text{ SetRefundRequested}, a_2 \text{ UnSetRefundRequested}, a_3 \text{ WithdrawRefund}, a_4 \text{ WithdrawDisputed}, a_5 \text{ SubmitResult}, a_6 \text{ AuthorizeRefund}\}.$$

The validator is then a partial labelled transition system. Each a carries three independent guard families: a signature predicate, a state predicate on σ and h , and a temporal predicate on the transaction’s validity interval. We write $\text{after}(t)$ for “the interval starts after t ” and $\text{before}(t)$ for “the interval ends before t ”. Separately, each a may impose output constraints, which are rules about where the spent value must go.

That separation is where the result of this paper lives. A redeemer can be tightly guarded about *who* may call it and say nothing at all about *where the money goes*.

The protocol fee is φ per mille. On mainnet $\varphi = 50$. We measured it at exactly 50.000 per mille of the token quantity in 8 of 8 observed a_0 spends, with the collateral return carrying zero tokens in all 8.

2 The guard table

Derived from the validator source, then checked by submitting transactions against the deployed bytecode. The four redeemers that bear on the result are below; a_1 , a_2 and a_5 move between states without exiting the contract and are given where they are used, in the proof of Theorem 1.

a	signature	states	on h	time	output constraints
a_0	$\text{sig}(s)$	RS	$h \neq \varepsilon$	$\text{after}(t_0)$	yes. Per asset j , at least $\lfloor q_j \varphi / 1000 \rfloor$ to the fee address, and at least c lovelace to b , at datum-tagged outputs. The remainder is unconstrained
a_3	$\text{sig}(b)$	FL, RR	$h = \varepsilon$	$\text{after}(t_1)$	none
a_4	k of A	D	$h \neq \varepsilon$	$\text{after}(t_2)$	none
a_6	$\text{sig}(s)$	D	—	$\text{after}(\theta_s)$	continuing output with $\sigma' = \text{RR}$, $h' = \varepsilon$, value preserved

Two remarks, and both matter for what follows. First, the arbitration branch a_4 constrains no output. The quorum may distribute the value as it chooses. In the successor version of this validator the split is lifted into the redeemer itself, as two explicit amount fields. The contract is not missing the notion of a negotiated split, and we do not claim it is.

Second, a_3 also constrains no output. In the successor version the source documents this as deliberate: a payout-binding address may be set when the escrow is created, and when it is left unset the buyer signs the destination. We are not the first to observe the property. We are the first to use it for a settlement between the two parties.

3 Arbitration closure

Theorem 1. *Let u be an escrow with $\sigma = D$ and $h \neq \varepsilon$, and suppose the present moment is past t_1 , so that no transaction can carry a validity interval ending before t_1 . Let u' be the continuing escrow produced by a seller-signed a_6 . Then the redeemers admissible on u' are exactly a_2 and a_3 . Both require $\text{sig}(b)$, and a_3 is the only one that removes value from the script. In particular a_4 is inadmissible on u' , and no sequence of transitions from u' restores it.*

In plain terms. Once the seller's delivery deadline has passed and the seller agrees to refund, the arbitrators are locked out for good. From that moment only the buyer can move the escrow at all, and the branch that takes the money out says nothing about where it goes.

Proof. By the a_6 output rule, u' has $\sigma' = RR$ and $h' = \varepsilon$. Take each redeemer in turn.

a	status on u'
a_0	inadmissible: requires $\sigma = RS$
a_1	inadmissible: its admissible states are FL, RS and D, and $\sigma' = RR$
a_2	admissible , buyer-signed. It preserves h and yields $\sigma = FL$, because $h' = \varepsilon$
a_3	admissible , buyer-signed: $\sigma' \in \{FL, RR\}$, $h' = \varepsilon$, past t_1 . It is the only branch here with no continuing output, so the only one that removes value
a_4	inadmissible: requires $\sigma = D$ and $h \neq \varepsilon$, and both are false
a_5	inadmissible: its state guard is vacuous, so only time and h can block it. It requires before (t_1), which no transaction can satisfy past t_1 , or $h \neq \varepsilon$ together with before (t_2), and $h' = \varepsilon$ falsifies that
a_6	inadmissible: requires $\sigma = D$

Both admissible transitions are buyer-signed, so no party other than the buyer can move u' at all.

It remains to show that no *sequence* restores a_4 . The only redeemer that writes a non-empty h is a_5 , and the argument above blocks a_5 on *every* state once t_1 is past, not only on RR: its state guard is vacuous, so the two temporal disjuncts are the whole of its guard, and the second needs the h it would be writing to be non-empty already. Hence $h = \varepsilon$ is invariant along every transition sequence from u' , including through FL after a_2 and back through a_1 . Since a_4 requires $h \neq \varepsilon$, it is unreachable. $\square$

Why the hypothesis is not a technicality. Before t_1 the theorem is false, and interestingly so. From RR with $h = \varepsilon$, a seller-signed a_5 with **before**(t_1) writes a non-empty h and, because the current state is neither FL nor RS, sets the new state to D. That is exactly the pair a_4 needs, restored in one step. Arbitration closure is a statement about escrows whose delivery deadline has passed. Every escrow we measured on mainnet, and every one we settled on preprod, is past t_1 ; indeed a_3 itself cannot be called before it.

Corollary (at source-reading strength only). The same holds in version 2 of the validator. There a_6 writes a distinct state and still empties h , while a_4 still requires $\sigma = D$ and $h \neq \varepsilon$. The result is not an artefact of one deployment. In version 2 the exit a_3 is unconstrained if and only if the optional payout-binding address is unset. Version 2 has never been used on mainnet and we executed nothing against it, so every V2 statement here is *the validator's source says*. For V1 the refusals were observed on chain.

What Theorem 1 does not say. It does not say the seller is cheated. A concession is voluntary, and the four instances we observed on mainnet conceded a full refund. What it says is narrower and harder: after a concession the value moves only if the buyer acts, and no third party can compel the outcome. Those four have been unmoved for 81 days.

4 Fee incidence depends on the exit path

Two orderings reach a settlement from D.

Path A. The buyer plays a_2 , so $\sigma \rightarrow RS$. The seller then exits with a_0 , paying the buyer a share.

Path B. The seller plays a_6 , so $\sigma \rightarrow RR$ and $h \rightarrow \varepsilon$. The buyer then exits with a_3 , paying the seller a share.

Exiting through a_0 costs, per asset,

$$\text{fee}_j^A = \left\lfloor \frac{q_j \varphi}{1000} \right\rfloor \leq \frac{q_j \varphi}{1000}, \quad \text{while} \quad \text{fee}_j^B = 0.$$

At $\varphi = 50$ the pair keeps up to 5 % of each asset by settling through path B instead of path A, and exactly 5 % whenever 1000 divides $q_j \varphi$. This is a first-order term and it favours B without ambiguity.

The protective floor behaves the same way. Path A guarantees the buyer c lovelace and no token. Path B guarantees nothing at all. On the 61 mainnet escrows currently in D, the contested quantity is overwhelmingly the token: 498.15 USDM against 54.31 ADA once collateral is removed. The one protective rule in the contract therefore protects almost none of what is actually in dispute.

5 Almost no split is self-enforcing

An agreement assigns the first mover a share $\alpha_j \in [0, 1]$ of each asset j , so the second mover is to receive $(1 - \alpha_j)q_j$. The second mover signs leg 2 and therefore holds a unilateral option: nothing in the validator conditions admissibility on the agreement, because the agreement has no on-chain representation. The question is when honouring it costs the second mover nothing.

Theorem 2. Fix a path and consider the second mover's choice at leg 2.

1. **Path B.** The exit a_3 has no output constraints, so the buyer's best deviation pays them the whole pot. Honouring is weakly optimal only if $\alpha_j = 0$ for every j .
2. **Path A.** The exit a_0 already compels the seller to send at least c lovelace to the buyer and fee_j^A of each asset to the fee address. Honouring therefore costs the seller nothing exactly when the agreement asks for no more than those mandatory outputs already deliver to the buyer, that is when

$$\alpha_L q_L \leq c \quad \text{and} \quad \alpha_j = 0 \quad \text{for every } j \neq L.$$

In particular no agreement that pays the first mover any token, or more lovelace than the collateral floor, is self-enforcing on either path.

In plain terms. Whoever signs second can always take everything the contract does not force them to give up. On path B the contract forces nothing, so only a split that gives the first mover nothing survives. On path A it forces a small lovelace floor, so the surviving splits are the ones already inside that floor.

Proof. Both cases compare two admissible transactions available to the second mover at leg 2 with identical inputs.

Path B. Deviating, the buyer pays q_j to themselves for every j ; honouring, they pay $(1 - \alpha_j)q_j$ to themselves and $\alpha_j q_j$ to the seller. Both transactions satisfy a_3 , which imposes no output rule, so the deviation is admissible and weakly dominates unless $\alpha_j q_j = 0$ for every j .

Path A. Both transactions must carry the tagged fee outputs and the c lovelace to the buyer, so those amounts are common to the two branches and cancel. What remains is the *additional* transfer to the buyer the agreement asks for, namely $\max(0, \alpha_L q_L - c)$ in lovelace and $\alpha_j q_j$ in each token. The deviation withholds exactly that, and it is admissible because a_0 leaves the remainder unconstrained. So honouring is weakly optimal precisely when that additional transfer is zero, which is the stated condition. $\square$

The contract therefore makes every negotiated split worth negotiating unreachable by incentives alone. That is the sharpest statement of the problem. It is also the statement of what has to be built.

6 Reachability by construction

Let T_1 be leg 1 and T_2 be leg 2. On Cardano a transaction identifier is a hash of its body. So $\text{id}(T_1)$ is fixed as soon as T_1 is built, before it is submitted. T_2 may therefore spend the output $\text{id}(T_1)\#0$, which does not yet exist, and may be signed against it.

The construction is this. Build T_1 and T_2 together. Have the second mover sign T_2 . Only then does the first mover sign and submit T_1 , with T_2 following immediately.

Refusal is removed. A signature on an unsubmitted transaction transfers nothing, so the second mover gives up nothing by signing first. Once T_2 is signed there is no act of refusal left to perform.

Input ownership is not a detail. The fee and collateral inputs of T_2 must belong to the conceding party. If they belonged to the second mover, that party could invalidate T_2 at no cost by spending them elsewhere, while keeping the option to take everything later.

What this does not provide. Pre-signing removes a refusal. It does not remove a front-run. Once T_1 is visible, the second mover may construct and broadcast a competing exit.

We submit both legs chained to one node. They landed in the same block in 5 runs of 5: blocks 5259678, 5259750, 5259751, 5259754 and 5259799, with leg 2 acknowledged 0.5 to 0.7s after leg 1 was sent. An adversary ran in four of those five. In none of them did it obtain a clean window, so we report no front-run measurement and the solver prices the event at its worst case.

The exposure is bounded below by the submitter's own gap between legs, measured at 249 to 296 ms in the contended runs, against an adversary whose first sighting came as early as 314 ms. Those are the same order of magnitude. We make no safety claim.

7 The bargaining set, priced from observation

Each party's reservation value is the value of not settling, which is the value of arbitration.

The seller's reservation value is zero, and that is measured. Across all 120 lifetime a_4 transactions on mainnet, the seller received nothing in any asset: 0 of 120. So $r_s = 0$, and any strictly positive share is individually rational for the seller.

The buyer's side cannot be given a value, only a break-even. The measured arbitration split is $\pi_{\text{ADA}} = 1.000$ and $\pi_{\text{tok}} = 0.736$ of the pot: the fraction the buyer received, per asset, in those 120 settlements. Model the arrival of the *next* arbitration as a Poisson process of rate λ . No a_4 transaction has occurred in $T = 313$ days, which gives the one-sided 95 % upper confidence bound

$$e^{-\lambda T} = 0.05 \implies \bar{\lambda} = \frac{\ln 20}{T} \approx \frac{1}{104 \text{ days}},$$

the exponential form of the rule of three, since $\ln 20 = 2.996$. This bounds the rate from above and does nothing else. The point estimate is zero and no lower bound exists. We fit only the silent window, not the lifetime history: the 120 events occurred under a regime that visibly ended, and pooling them would assume the homogeneity the silence is evidence against.

It is therefore not admissible to turn $\bar{\lambda}$ into a value of waiting and subtract it from the pot. There are two reasons, and the second is the binding one.

1. A buyer facing a **deadline** H has a well-defined claim $\pi_j(1 - e^{-\lambda H})$. At $\bar{\lambda}$ that evaluates to 6.5 %, 25.0 % and 57.7 % of their arbitration payoff for $H = 7, 30, 90$ days. But it is an upper bound computed from an upper bound, about a deadline we never observed.
2. A buyer who can **wait indefinitely**, against an offer that stays open, gives up nothing by waiting. Such a buyer accepts less than the arbitration payoff only through a time preference, and nothing in the chain data measures a discount rate. At a plausible 10 % per year, the same 95 % bound makes waiting worth about 97 % of the ADA.

So we publish break-evens, not values. Let β_j be the share of asset j offered *to the buyer*, and suppose the buyer has a deadline H . If $\beta_j \geq \pi_j$ the offer already matches the best arbitration has ever paid, and accepting weakly dominates waiting at every rate. Otherwise $0 \leq \beta_j < \pi_j$ and waiting beats accepting if and only if

$$\lambda > \lambda^*(\beta_j, H) = -\frac{1}{H} \ln\left(1 - \frac{\beta_j}{\pi_j}\right) > 0,$$

with the break-even horizon H^* following by inversion. The solver reports λ^* and H^* next to the measured bound $\bar{\lambda}$. The reader supplies the belief; we do not supply it for them.

The acceptable band is conditional and must always be quoted with its horizon. Writing it as the *seller's* share $1 - \beta_j$, a buyer with deadline H accepts exactly

$$\mathcal{B}_j(H) = \left[0, 1 - \pi_j \left(1 - e^{-\bar{\lambda} H}\right)\right],$$

non-empty for every H because $\pi_j \leq 1$. On path A the seller must additionally fund the mandatory outputs, so the ceiling falls by $\varphi/1000$ in every asset and by a further c/q_L in lovelace. **No split is called rational in this paper without a stated horizon or a stated discount rate.**

One further term keeps settlement attractive to both parties even if arbitration were certain to arrive. In the 120 observed arbitrations, 26.4 % of the token pot reached neither party. We keep it as a solver term and do not publish it as an output. The gross flows to admin-controlled addresses include those wallets' own change, so the only defensible quantity is the pot minus what the parties received.

8 What was executed

Every structural claim above was checked against the deployed bytecode by submitting transactions on preprod. Its script hash is byte-identical to mainnet's.

Six settlements, twelve legs, no admin key in any of them. Each leg's CBOR was read back from the chain and its hash checked. Across all twelve, every vkey witness hashes to either the buyer or the seller in that escrow's own datum. None of the three deployed admin key hashes appears anywhere.

The clearest single example settles a token pot in block 5259570 through the file-drop path:

Leg 1 a_6 `AuthorizeRefund`
be1a2161b8c451309549265337893cc05bd4f75e9a7e3b971ccca86d7bcb4df8

Leg 2 a_3 `WithdrawRefund`
ccb04dd233010d1e71ca0ebacb68cc83c28247e78e16b5f84a096389b2eab637

Leg 1 spends the escrow in D and returns the whole pot to the script in RR, signed by the seller alone. Leg 2 spends `leg1#0` and the value leaves the script: the buyer is the required signer, the seller co-signs because their own UTxO pays the fee. The split is 12 tADA and 6 tUSDM to the buyer, 8 and 4 to the seller. Other settlements in the same set run 70/30 and 25/75, so the construction is not tied to one ratio.

We also executed the admin pair on our **own** deployment of the same compiled code. There a_4 is accepted before a concession and refused after it, with the state and hash guards named in the failure. And we executed negative controls, in which the engine predicts a refusal, the transaction is submitted anyway, and the validator refuses it.

Claims that rest on reading rather than execution are labelled as such throughout the repository. On the shared mainnet script we are not an admin and never will be, so the admin branch there remains *the validator's source says*, never *we observed*.

No transaction was ever built, evaluated or submitted against a mainnet escrow.
Mainnet is read-only in this work.

9 Limitations

1. **The front-run is unmeasured**, not measured and negative. Four contended attempts failed for transport reasons, which we diagnose rather than hide.
2. **The arrival-rate bound is a bound.** $\bar{\lambda}$ is a 95 % upper limit from zero observations in the silent window. It is not an estimate, and the true rate may be far lower.
3. **The dispute rate is not computable from our data.** The contract has 51,329 lifetime transactions, but one job spends several, so the job denominator is unknown. We report counts, 120 arbitrated and 61 open, and no rate.
4. **The model assumes the parties already agree on the facts.** Nothing here decides who was right, and nothing here could. The deliverable is private and only its hash is public. That is the premise, not a gap.
5. **The deployed V1 bytecode is not reproducible** from the current public source. Compiling with the toolchain named in the committed blueprint reproduces the unapplied hash exactly, but the applied script is 6,003 bytes against 6,323 on chain. The V2 deployment reproduces byte-exactly, which establishes the method and isolates the gap to a code difference rather than to our parameters.